

Parallel and Distributed Programming

OpenMP Programming Assignment

Total: 100 points

Complete the following three problems using **C or C++ and OpenMP**. Each problem requires both a serial implementation and parallel implementations, together with correctness checks and a short performance analysis.

General Requirements

Submission

For each problem, submit:

- Source code for the serial and required parallel implementations.
- A `Makefile` that compiles all programs for the problem.
- A `README` explaining how to compile and run the programs, the input parameters, and the meaning of the output.
- A short experimental report containing correctness results, timing measurements, speedups, and answers to the discussion questions.

All submitted programs must compile and run successfully.

Experimental Setup

Report your processor model, number of CPU cores, compiler version, and compilation flags.

Use identical inputs and consistent compiler optimization settings for the serial and parallel implementations. Run each configuration at least **three times** and report the **median execution time**.

Calculate speedup as

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)},$$

where T_{serial} is the execution time of the serial implementation and $T_{\text{parallel}}(p)$ is the execution time of the parallel implementation using p threads. **Do not substitute the one-thread execution time of the parallel implementation for the serial baseline.**

Exclude input generation, file input/output, and correctness checking from the timed region. Include synchronization, initialization of thread-private statistics, and result merging when these operations are required by the computation.

Problem 1: Parallel Grayscale Histogram (30 points)

Problem Description

Each pixel in a grayscale image is represented by an integer from 0 to 255. A grayscale histogram counts how many pixels have each possible intensity value.

In this problem, an image is represented by a one-dimensional array `pixels` containing N elements. You do not need to read an actual image file. Compute an array `hist` of length 256 such that

$$\text{hist}[v] = |\{i \mid 0 \leq i < N, \text{pixels}[i] = v\}|, \quad 0 \leq v < 256.$$

The following pseudocode illustrates the serial computation:

```
// Pseudocode. Use 64-bit integer counters for hist.
for (int v = 0; v < 256; ++v) {
    hist[v] = 0;
}

for (size_t i = 0; i < N; ++i) {
    hist[pixels[i]]++;
}
```

For example, given

```
pixels = [0, 1, 1, 2, 2, 2, 255, 255]
```

the nonzero histogram entries are

```
hist[0]   = 1
hist[1]   = 2
hist[2]   = 3
hist[255] = 2
```

Programming Requirements

Implement the serial version and the following two parallel versions.

Version A: Shared Histogram. All threads update a single shared `hist` array. Use the OpenMP `atomic` construct to ensure that counter updates are correct.

Version B: Thread-Private Histograms. Each thread maintains its own local histogram. After processing the input, merge the local histograms into the final result. Do not update the global histogram for every input element in this version.

Your program must accept the array length and thread count as parameters. Check that every histogram entry exactly matches the serial result. Also verify that

$$\sum_{v=0}^{255} \text{hist}[v] = N.$$

Testing and Analysis

For performance experiments, test at least $N = 10^6$ and $N = 10^7$ with both of the following input distributions:

- **Uniform random input:** Generate each element uniformly from 0 to 255. Use a fixed random seed and generate the input before timing begins.
- **Concentrated input:** Set every element to 128.

Correctness tests must also include an array of length 1 and an array whose length is not divisible by the thread count.

In your report, answer the following questions:

1. Why might simply adding a parallel loop directive to the serial loop produce incorrect results?
2. How do the two parallel versions perform under the two input distributions? Explain the observed differences.
3. What additional work does the thread-private approach introduce?

Hint: Threads that process different input positions may still update the same output position.

Problem 2: Parallel Temperature Iteration on a Two-Dimensional Plate (35 points)

Problem Description

Represent the temperature of a plate using an $N \times N$ grid, where $N \geq 3$.

Initialize the grid and its fixed boundaries as follows:

- The top boundary has temperature 100, except at its two corner points.
- All other boundary points, including all four corners, have temperature 0.
- All interior points initially have temperature 0.

During each iteration, the new temperature at an interior point is the average of the temperatures at its four neighboring points in the **previous iteration**:

$$U_{i,j}^{(t+1)} = \frac{U_{i-1,j}^{(t)} + U_{i+1,j}^{(t)} + U_{i,j-1}^{(t)} + U_{i,j+1}^{(t)}}{4}, \quad 1 \leq i, j \leq N-2.$$

Boundary temperatures remain unchanged. Perform exactly T iterations, where $T \geq 1$. Do not terminate early based on convergence.

For each iteration, also compute the maximum absolute temperature change:

$$\Delta^{(t+1)} = \max_{1 \leq i,j \leq N-2} |U_{i,j}^{(t+1)} - U_{i,j}^{(t)}|.$$

The following pseudocode illustrates the serial computation:

```
// Pseudocode. A and B are N-by-N grids of double values.
// initialize sets the fixed boundaries and zeros the interior.
initialize(A);
initialize(B);

for (int t = 0; t < T; ++t) {
    delta = 0.0;

    for (int i = 1; i < N - 1; ++i) {
        for (int j = 1; j < N - 1; ++j) {
            B[i][j] = 0.25 * (
                A[i - 1][j] + A[i + 1][j] +
                A[i][j - 1] + A[i][j + 1]
            );

            delta = max(delta, fabs(B[i][j] - A[i][j]));
        }
    }

    swap(A, B); // Exchange the roles of the two buffers.
}
```

For example, when $N = 4$ and $T = 1$, the final temperature grid is

0	100	100	0
0	25	25	0
0	0	0	0
0	0	0	0

The maximum temperature change in this iteration is 25.

Programming Requirements

Implement a serial version and an OpenMP parallel version. Parallelize the computation of interior grid points within each iteration, and use an OpenMP reduction to compute the maximum temperature change.

Every new temperature must be computed using only values from the previous iteration. Do not mix previous-iteration values with values already updated during the current iteration. You may use two grid buffers and exchange their roles after each iteration.

Your program must accept the grid size, iteration count, and thread count as parameters. Output the maximum temperature change in the final iteration, the execution time, and the maximum absolute difference between the serial and parallel results.

Use double precision. The final grids must satisfy

$$\max_{i,j} |U_{i,j}^{\text{parallel}} - U_{i,j}^{\text{serial}}| \leq 10^{-8}.$$

Testing and Analysis

For performance experiments, test at least

$$N \in \{128, 256, 512\}, \quad T = 1000.$$

Correctness tests must also include small grids with $N = 3$ and $N = 4$, and iteration counts $T = 1$ and $T = 2$.

In your report, answer the following questions:

1. Why can different iterations not be executed directly in parallel?
2. Why might updating the temperature grid in place change the result?
3. Which operations require synchronization at the end of each iteration?
4. How does parallel overhead affect execution time when the grid is small?

Hint: Interior points within one iteration can be computed independently, but the next iteration requires the complete results of the current iteration.

Problem 3: Mandelbrot Computation and OpenMP Scheduling (35 points)

Problem Description

In this problem, you will compute an iteration count for each pixel in a two-dimensional image using a simple recurrence.

For a complex number $c = x + yi$, start from $z_0 = 0$ and repeatedly compute

$$z_{k+1} = z_k^2 + c.$$

Stop when $|z_k|^2 > 4$ or when K iterations have been completed. Record the number of iterations actually performed as the output value of the pixel.

You do not need a complex-number library. Writing $z = a + bi$, one update becomes

$$a_{\text{new}} = a^2 - b^2 + x, \quad b_{\text{new}} = 2ab + y.$$

Both right-hand sides must use the values of a and b **before** the update.

Let the image width be W and its height be H , with $W, H \geq 2$. The pixel at row i and column j corresponds to

$$x = -2 + \frac{3j}{W-1}, \quad y = -1.5 + \frac{3i}{H-1},$$

where $0 \leq i < H$ and $0 \leq j < W$. Use floating-point division when computing these coordinates.

The following pseudocode computes one pixel:

```
// Pseudocode. x, y, a, b, and next_a use double precision.
a = 0.0;
b = 0.0;
count = 0;

while (a * a + b * b <= 4.0 && count < K) {
    next_a = a * a - b * b + x;
    b = 2.0 * a * b + y;
    a = next_a;
    count++;
}

iterations[i][j] = count;
```

For example, the iteration count for the point $c = 0$ is K . For the point $c = 1$, the iteration count is 3 when $K \geq 3$.

Programming Requirements

Implement a serial version and OpenMP parallel versions that parallelize the **outer loop over image rows**. Each loop iteration must compute one complete row so that the unit of work remains the same across versions.

Implement and compare the following three scheduling configurations:

```
schedule(static)
schedule(dynamic, 1)
schedule(guided, 1)
```

For `dynamic, 1`, each chunk contains one row. For `guided, 1`, the minimum chunk size is one row. Use the same pixel computation in all three versions.

Your program must accept the image dimensions, maximum iteration count, and thread count as parameters. The iteration-count matrix produced by each parallel version must match the serial result **exactly at every pixel**.

For each thread, also record the number of pixels it processed and the sum of the iteration counts for those pixels. Accumulate these statistics locally and write them back after processing. Do not introduce a global lock for every pixel merely to collect statistics. You do not need to print the complete matrix for large images.

Testing and Analysis

For performance experiments, test at least

$$(W, H) \in \{(512, 512), (1024, 1024)\}, \quad K = 500.$$

Correctness tests must also include an image whose height is not divisible by the thread count.

Report execution times, speedups, and the total iteration count handled by each thread for all three scheduling configurations.

In your report, answer the following questions:

1. Why can threads that process the same number of rows perform different amounts of computation?
2. Which scheduling configuration performs best in your experiments? Explain your answer using the measured thread workloads and execution times.
3. Does a more flexible scheduling strategy always reduce execution time? Why or why not?

Hint: Different pixels may require different numbers of iterations before reaching a stopping condition. The number of pixels alone is therefore not a sufficient measure of computational workload.

Grading Rubric

Problem	Main Topics	Point Allocation
Problem 1: Grayscale Histogram	Data races, atomic updates, thread-private data, result merging	Correctness: 10; two parallel implementations: 12; experimental analysis: 8
Problem 2: Temperature Iteration	Iteration dependencies, double buffering, synchronization, maximum reduction	Correctness: 15; parallel implementation and synchronization: 12; experimental analysis: 8
Problem 3: Mandelbrot Computation	Loop parallelism, scheduling, workload measurement, load balancing	Correctness: 12; three scheduling configurations and workload statistics: 13; experimental analysis: 10

A specific speedup is not required for full credit. Correct programs, appropriate parallel implementations, and well-supported explanations of limited speedup or performance degradation will receive the corresponding credit.